

# CAP theorem and Big Data Ecosystem evaluation

MASTER DEGREE IN COMPUTER SCIENCE

Big Data Analytics – Academic Year 2025/2026

**Author:**

Pellacani Simone (ID: 217424)

**Repository:**

[https://github.com/s0SimoneP0s/BD\\_assignements/BD](https://github.com/s0SimoneP0s/BD_assignements/BD)

11 settembre 2026

**UNIMORE**  
UNIVERSITÀ DEGLI STUDI DI  
MODENA E REGGIO EMILIA



# Sommario

Questo documento presenta una valutazione sperimentale completa di un sistema distribuito NoSQL inizialmente basato su Redis. La ricerca ha seguito un percorso strutturato in tre fasi incrementalmente partendo da pipeline di esempio e sui concreti.

Inizialmente si è testato il comportamento del sistema sotto guasti controllati e partizioni di rete, analizzando le osservazioni alla luce del teorema CAP per verificare la stabilità necessaria alla gestione di grandi volumi di dati critici.

Nella seconda fase, abbiamo incrementalmente aggiunto FalkorDB (Redis-based) per sostituire Neo4j in una pipeline esistente, dimostrando che FalkorDB può mantenere ottime garanzie di stabilità del sistema, ma ha differenti approcci per la gestione dei dati e delle transazioni rispetto a Neo4j.

Per garantire la stabilità del sistema, sono stati condotti test approfonditi che simulano guasti e partizioni di rete, monitorando attentamente il comportamento dei nodi e la consistenza dei dati.

Il codice dei test e gli script di orchestrazione sono disponibili nel repository del progetto e su [https://github.com/s0SimoneP0s/BD\\_assignments](https://github.com/s0SimoneP0s/BD_assignments).

## Indice

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>Sommario</b>                                             | <b>1</b> |
| <b>1 Requisiti</b>                                          | <b>2</b> |
| <b>2 Obiettivi</b>                                          | <b>2</b> |
| <b>3 Ambito sperimentale e ipotesi CAP</b>                  | <b>2</b> |
| <b>4 Redis data model and CAP implications</b>              | <b>2</b> |
| 4.1 Panoramica                                              | 2        |
| 4.2 Modelli di dati in Redis                                | 2        |
| 4.3 Architettura Redis di riferimento                       | 3        |
| 4.4 Replicazione, persistenza e implicazioni di consistenza | 4        |
| 4.5 Atomicità                                               | 4        |
| 4.6 Cluster Sharding                                        | 4        |
| 4.7 Persistenza                                             | 4        |
| 4.8 Test setup sperimentale                                 | 4        |
| 4.9 Test generico                                           | 4        |
| 4.10 API e workload                                         | 5        |
| 4.10.1 Dettagli dei test                                    | 5        |
| <b>5 FalkorDB</b>                                           | <b>5</b> |
| 5.1 Sintassi supportata                                     | 5        |
| 5.2 API, driver e algoritmi                                 | 6        |
| 5.3 Confronto sintassi Neo4j vs FalkorDB                    | 6        |
| 5.3.1 Connessione e inizializzazione driver                 | 6        |
| 5.4 Gestione grafo                                          | 6        |
| 5.5 Rappresentazione GraphBLAS                              | 6        |
| 5.6 Projections                                             | 7        |
| 5.7 Caricamento batch e parametri                           | 7        |
| 5.8 Effort e portabilità                                    | 7        |
| <b>6 Scalare dati tabulari con Spark</b>                    | <b>8</b> |
| 6.1 Feature principali                                      | 8        |
| 6.1.1 Utilizzi comuni                                       | 8        |
| 6.2 Tipi di dato                                            | 8        |
| 6.2.1 Rappresentazioni                                      | 8        |
| 6.2.2 Tidy data e formato wide/long                         | 9        |
| 6.3 Librerie avanzate                                       | 9        |
| 6.4 Effort di conversione da pandas a Spark DataFrame       | 9        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>7 Usiamo Sail</b>                                | <b>9</b>  |
| 7.1 Computing . . . . .                             | 10        |
| 7.2 Benchmark della sessione Sail . . . . .         | 10        |
| 7.3 Nota sull'integrazione nella pipeline . . . . . | 10        |
| 7.4 Limiti rispetto all'ecosistema Spark . . . . .  | 10        |
| <b>8 Breve riepilogo</b>                            | <b>11</b> |
| 8.1 Backend Redis: CAP Theorem . . . . .            | 11        |
| 8.2 Graph database Falkordb . . . . .               | 11        |
| 8.3 Spark computing engine . . . . .                | 11        |
| 8.4 Sail tabular computing . . . . .                | 11        |
| <b>9 Discussione e conclusioni</b>                  | <b>11</b> |
| <b>Riferimenti</b>                                  | <b>12</b> |

# 1 Requisiti

- **Funzionali:** deploy container multinodo capace di accettare letture e scritture concorrenti; accesso al cluster tramite localhost per eseguire workload e iniettare guasti.

# 2 Obiettivi

- Misurare i compromessi tra disponibilità e consistenza durante partizioni se evidenziati.
- Fornire test ripetibili che simulino guasti e producano log leggibili dalla macchina.
- Riportare la correttezza (violazioni di consistenza) e metriche operative.
- Provare la portabilità del codice Neo4j verso FalkorDB, dimostrando che la pipeline dati può essere eseguita con un comportamento simile con un effort controllato.
- Scalare ulteriormente la computazione utilizzando Spark.

# 3 Ambito sperimentale e ipotesi CAP

Il **teorema CAP** afferma che un sistema dati distribuito può fornire al massimo due delle seguenti tre garanzie simultaneamente: Consistenza, Disponibilità e Tolleranza alle partizioni [1]. In pratica: [14]

- **Disponibilità** (availability): ogni richiesta a un nodo non fallito riceve una risposta (senza garantire sia il valore più recente). verificheremo se il sistema continua ad accettare e servire richieste corrette quando alcuni nodi falliscono.
- **Consistenza** (copy-consistency): tutti i nodi vedano gli stessi dati nello stesso momento. verificheremo se le repliche ritornano lo stesso stato dopo eventi casuali sui nodi di rete.
- **Tolleranza alle partizioni** (partitioning): il sistema continua a operare nonostante partizioni di rete arbitrarie. verificheremo se il sistema continua a operare anche quando le partizioni di rete vengono forzate.

Gli esperimenti sono condotti in un contesto ideale: un piccolo cluster containerizzato. I test sono progettati per misurare disponibilità, diverse nozioni di consistenza e comportamento di recupero.

# 4 Redis data model and CAP implications

Redis è un sistema ricco di funzionalità, ma nel contesto di questo lavoro si presta a un uso semplice: è facile da deployare, configurare e integrare nel cluster di test. In particolare, useremo solo la parte più essenziale, il **String key-value**.

## 4.1 Panoramica

Redis è principalmente uno store key-value che espone diversi tipi di dato nativi [12]. Ogni tipo offre garanzie semantiche e pattern d'uso differenti; queste scelte influenzano replicazione, concorrenza e recupero. Abbiamo scelto appositamente questo strumento perché unisce un insieme molto ricco di funzionalità a un uso pratico semplice sia a livello implementativo, sia nella fase di deploy sia durante la configurazione del sistema.

## 4.2 Modelli di dati in Redis

L'obiettivo non è mostrare come anche il modello influenzi la correttezza dell'applicazione, ma guidare la progettazione dei test per verificarli adeguatamente in un caso base.

Idealmente il sistema gestisce separatamente le strutture dati dalla logica di partizionamento e replicazione, quindi verrà testato il comportamento di una struttura dati campione.

Qua a seguito una veloce panoramica dei principali tipi di dato in Redis, con esempi di utilizzo e pattern d'uso tipici utilizzando API python.

- **Stringhe:** sono la scelta corretta per valori atomici, contatori, cache di piccole dimensioni e flag di stato. Risolvono il problema del salvataggio semplice chiave-valore e supportano operazioni rapide e atomiche come aggiornamenti, letture e incrementi.

```
r.set("user:1:name", "Mario")
name = r.get("user:1:name")
r.incr("page:views")
```

- **Hash:** sono utili quando un'entità ha molti attributi, ad esempio un profilo utente, un record prodotto o una configurazione. Risolvono il problema di aggiornare solo alcuni campi senza riscrivere tutto l'oggetto.

```
r.hset("user:1", mapping={"name": "Mario", "age": 22})
user_name = r.hget("user:1", "name")
r.hincrby("user:1", "age", 1)
```

- **Liste:** sono adatte a code di lavoro, feed temporali, log append-only e buffer di messaggi. Risolvono il problema dell'ordinamento degli elementi e dell'elaborazione FIFO/LIFO.

```
r.lpush("jobs", "job-1")
next_job = r.rpop("jobs")
size = r.llen("jobs")
```

- **Set:** sono indicate per rappresentare collezioni di elementi unici, come tag, utenti online o insiemi di appartenenza. Risolvono il problema dei duplicati e rendono efficienti test di membership e operazioni insiemistiche.

```
r.sadd("online-users", "alice")
is_member = r.sismember("online-users", "alice")
members = r.smembers("online-users")
```

- **Sorted set (zset):** sono utili per classifiche, leaderboard, priorità e range di punteggio, ad esempio top utenti o job scheduling. Risolvono il problema dell'ordinamento dinamico con punteggi aggiornabili.

```
r.zadd("leaderboard", {"alice": 120, "bob": 95})
top = r.zrevrange("leaderboard", 0, 2, withscores=True)
score = r.zscore("leaderboard", "alice")
```

- **Stream:** sono adatti a pipeline di eventi, telemetria e processing distribuito con consumer group. Risolvono il problema dell'ingestione ordinata dei messaggi e della loro elaborazione affidabile.

```
msg_id = r.xadd("events", {"type": "login", "user": "alice"})
entries = r.xrange("events", count=1)
r.xgroup_create("events", "workers", id="0", mkstream=True)
```

- **Tipi specializzati:** bitmap, HyperLogLog e strutture geospaziali sono utili per casi come conteggi approssimati, tracciamento di flag compatti e query geografiche. Risolvono problemi di spazio, cardinalità approssimata e rappresentazione efficiente di dati derivati.

```
r.setbit("flags", 7, 1)
count = r.pfcount("unique-visitors")
r.geoadd("stores", {"store-1": (12.5, 41.9)})
```

Nel nostro esperimento, però, questi modelli non vengono esercitati tutti: il workload di test usa principalmente la struttura **String**, tramite operazioni SET/GET, perché l'obiettivo è misurare disponibilità, consistenza e tolleranza ai guasti del cluster nel caso più semplice e controllabile.

### 4.3 Architettura Redis di riferimento

Redis è un archivio dati in-memory che opera come database persistente tramite posix COW snapshot. La sua architettura è ottimizzata per garantire latenze estremamente ridotte, delegando la gestione della durabilità e della distribuzione a meccanismi configurabili.

## 4.4 Replicazione, persistenza e implicazioni di consistenza

In modalità cluster (a seconda della configurazione) le chiavi possono essere distribuite tra più nodi o accumulate nel primary e poi distribuite tra i master (sharded). Questo significa che, se un client deve lavorare su informazioni salvate su nodi diversi in configurazione sharding, le operazioni devono essere coordinate con attenzione oppure ricostruite lato client unendo i risultati ottenuti dai vari nodi.

Anche gli stream, cioè flussi ordinati di messaggi, e i consumer group, cioè gruppi di processi che leggono e gestiscono questi messaggi, introducono aspetti di ordinamento e distribuzione del carico. Per questo sono rilevanti quando si valuta se i messaggi vengono letti ed elaborati in modo coerente. Ciò dimostra che Redis non è solo un archivio semplice di coppie chiave-valore: quando i dati sono distribuiti su più nodi o quando entrano in gioco flussi di messaggi intra-cluster, tutt'altro che banali.

Nel test è stata usata una configurazione priva di sharding.

Conseguenze progettuali:

- Per un comportamento fortemente consistente, i client dovrebbero preferire letture dal primary oppure usare livelli supportati da consenso; in caso contrario, le letture dalle repliche possono essere "Soft state".
- Comandi atomici come HSET e LPUSH riducono alcuni problemi di concorrenza, ma non verranno trattati questi scenari. Per gestire carichi concorrenti il client python è stato gestito con un lock per serializzare le operazioni di scrittura e lettura.

## 4.5 Atomicità

Redis utilizza un'architettura **single-threaded** per l'elaborazione dei comandi. Questo semplifica la gestione della concorrenza: ogni comando viene eseguito in modo sequenziale e atomico, eliminando la necessità di lock complessi internamente al database e garantendo che le operazioni sulle strutture dati siano sempre consistenti a livello di singolo nodo.

## 4.6 Cluster Sharding

In configurazione Cluster Sharded, la distribuzione dei dati avviene tramite gli **hash slot**. Lo spazio delle chiavi è diviso in 16.384 slot distribuiti tra i nodi master tramite hash function. Questo meccanismo permette di scalare il sistema aggiungendo nodi e ridistribuendo gli slot in modo trasparente.

## 4.7 Persistenza

Per la durabilità dei dati, Redis offre due meccanismi:

- **RDB (Redis Database Backup)**: snapshot puntuali del dataset a intervalli definiti. È efficiente per i riavvii ma può comportare perdite di dati tra uno snapshot e l'altro.
- **AOF (Append-Only File)**: un log di tutte le operazioni di scrittura. Garantisce una persistenza maggiore (quasi real-time) a fronte di file di log più grandi. Soggetto attenzionato dal test di partizionamento.

## 4.8 Test setup sperimentale

I workload e le procedure automatizzate di test tra cui la metodologia è stata presa come esempio [11] vengono eseguiti tramite orchestrazione containerizzata (Docker Compose) per garantire riproducibilità e facilità di implementazione. I generatori di carico sono script Python personalizzati; gli eseguibili nella cartella `experiments` eseguono letture e scritture secondo gli scenari descritti sotto.

## 4.9 Test generico

- T: numero di thread concorrenti simulati (thread/processi). Aumentando questo valore si incrementa la concorrenza totale.
- N: numero di operazioni locali eseguite da ciascun thread; il totale delle operazioni è  $T \times N$ .
- P: numero di nodi nel cluster Redis; 3 nel nostro caso.

---

**Algorithm 1** Test generico

---

```
1: procedure TESTGENERICO( $P, T, N$ )
2:   Avviare cluster con  $P$  nodi
3:   Lanciare  $T$  thread worker
4:   for  $t = 1$  to  $T$  do
5:     Ogni worker esegue  $N$  operazioni locali e casuali:
6:     per ciascuna ne registra (timestamp, operazione, risultato)
7:   end for
8:   Attendere il completamento dei worker e raccogliere i log
9: end procedure
```

---

## 4.10 API e workload

Sotto una breve descrizione dei test e delle chiamate API usate per generare carico e misurare il comportamento del cluster Redis. Non sempre si tratta di chiamate esposte direttamente dal client Redis ufficiale, ma a volte sono state progettate apposite funzioni wrapper che orchestrano le operazioni e registrano i log.

- **start\_stop\_container**: script o controlli sui container che arrestano/riavviano i nodi e manipolano le regole di rete, ad esempio con `tc`, per creare partizioni.
- **scan\_all**: i client registrano lo stato del nodo, l'esito e il valore osservato per le letture.
- **random\_rw\_event**: esegue operazioni casuali di lettura e scrittura.
- **random\_partition\_event**: esegue eventi casuali di partizionamento della rete (spegne/riavvia nodi e legge/scrive dati nel frattempo).
- **random\_primary\_switch**: esegue eventi casuali di cambio dei nodi primari.

### 4.10.1 Dettagli dei test

I log grezzi relativi agli esperimenti sono raccolti nella cartella `experiments/logs/` del repository; il riepilogo delle esecuzioni batch è disponibile in `experiments/logs/stats.csv`. Per il repository dei log si può usare il percorso [https://github.com/s0SimoneP0s/BD\\_assignments/BD/experiments/logs](https://github.com/s0SimoneP0s/BD_assignments/BD/experiments/logs), dove viene mostrata solo la parte CSV.

- **Test di consistenza**: le esecuzioni con `test_consistency` verificano se il cluster mantiene un comportamento coerente sotto carico concorrente e durante i cambi di primario.
- **Test di disponibilità**: le esecuzioni con `test_availability` verificano se il cluster continua a rispondere alle operazioni di base anche in presenza di disturbi controllati.
- **Test di partizionamento**: le esecuzioni con `test_partitioning` osservano il comportamento del sistema quando la rete viene perturbata in modo controllato.

## 5 FalkorDB

FalkorDB è un modulo Redis: il motore grafo vive nel processo Redis ed estende il server con comandi dedicati. Il caricamento avviene tramite libreria dinamica (ad esempio `/usr/lib/redis/modules/falkordb.so`).

### 5.1 Sintassi supportata

Il linguaggio operativo combina Cypher e comandi Redis del modulo. Sono utilizzati pattern Cypher come `MATCH`, `MERGE`, `SET`, `UNWIND`, e comandi `GRAPH.QUERY`, `GRAPH.RO_QUERY`, `GRAPH.DELETE`, `GRAPH.LIST`, `GRAPH.EXPLAIN`, `GRAPH.PROFILE`. La progettazione della pipeline di esempio è come al solito idempotente nella gestione dell'esperimento.

```
GRAPH.QUERY social "MATCH_(n)_RETURN_count(n)"
GRAPH.RO_QUERY social "MATCH_(n)_RETURN_n_LIMIT_5"
GRAPH.EXPLAIN social "MATCH_(a)-[:KNOWS]->(b)_RETURN_a,b"
```

## 5.2 API, driver e algoritmi

La connessione avviene tramite client Redis con `ping()` e invio di comandi `GRAPH.*`. Le relazioni sono dirette, con tipo singolo; nodi e relazioni supportano proprietà. I tipi gestiti includono nodi, relazioni, path, scalari, stringhe, booleani, interi, float, geospaziali, null, array e mappe. FalkorDB distingue gli elementi strutturali del property graph (nodi e relazioni), i path ottenibili come risultati delle query e i valori scalari/collection utilizzabili nelle proprietà e nei risultati. Tra questi sono inclusi stringhe, valori booleani, numerici, valori geospaziali, array e mappe, con limitazioni specifiche per la persistenza delle mappe e delle collection.

## 5.3 Confronto sintassi Neo4j vs FalkorDB

La differenza pratica è nel livello di esecuzione: in Neo4j la query è nativa del server grafo, in FalkorDB la query è veicolata come comando Redis con nome grafo esplicito o viene eseguita la call verso API GraphBLAS.

### 5.3.1 Connessione e inizializzazione driver

La sostituzione del driver è diretta: Neo4j usa `GraphDatabase.driver(...)` con verifica di connettività, mentre FalkorDB usa `Redis(...)` con `ping()` e successivi comandi `GRAPH.*`. Vari esempi di codice sono disponibili nel repository del progetto, con riferimento particolare al Notebook `06-diff-API.ipynb`. Le particolari differenze si manifestano in output troncati in modi differenti, query con sintassi non simile e implementazione di differenti algoritmi di analisi del grafo.

## 5.4 Gestione grafo

In FalkorDB il nome del grafo è un identificatore operativo (es. `social`, `flight_network`) e viene passato in ogni comando. Nel contesto Redis Cluster, il nome del grafo determina hash slot e master shard di appartenenza; un singolo grafo risiede su un solo master in contesto di shard. La nostra configurazione è di tipo master-repliche. FalkorDB tratta ciascun grafo come una singola Redis key. Redis Cluster assegna la key a uno dei 16.384 hash slot; quindi il grafo viene assegnato a un solo shard. A livello FalkorDB/Redis Cluster, le repliche sono asincrone e possono essere utilizzate per failover e read scaling. Il deployment utilizza Redis Cluster con master e repliche. Ogni grafo appartiene a un singolo master shard; le repliche mantengono una copia asincrona dei dati del master, possono essere promosse in caso di failover se implementassimo un algoritmo di heartbeat e VIP dedicato.

## 5.5 Rappresentazione GraphBLAS

FalkorDB rappresenta internamente il grafo mediante matrici di adiacenza sparse e utilizza GraphBLAS per esprimere le operazioni di analisi del grafo. Questa rappresentazione è alla base degli algoritmi di pathfinding, centralità e community detection disponibili nel motore, tra cui BFS, PageRank, Betweenness Centrality, Harmonic Centrality, WCC e CDLP.

Un esempio di ottimizzazione di PathFinding usando matrici per un grafo orientato con archi  $a \rightarrow b$  e  $b \rightarrow c$ , abbiamo una matrice di adiacenza iniziale  $A$  e un'esistenza di un percorso può essere rappresentato come prodotto matriciale  $A^2$ .

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$
$$A^2 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

La matrice  $A^2$  rappresenta i cammini di lunghezza due: l'elemento  $(a, c)$  è diverso da zero perché esiste il cammino  $a \rightarrow b \rightarrow c$ .

La stessa formulazione può essere utilizzata per operazioni n-hop. Ad esempio, una BFS può essere espressa attraverso operazioni sparse matrice-vettore, applicate iterativamente alla matrice di adiacenza iniziale. In questo modo il traversal del grafo può essere ricondotto a primitive di algebra lineare sparse [4].

Mentre l'implementazione per una community detection in FalkorDB utilizza il label propagation [2].

La label propagation è un algoritmo per la ricerca di comunità, ma può dare risultati randomizzati a seconda dell'ordine di visita dei nodi e della gestione dei casi di parità [13].

La seguente suite di algoritmi è disponibile in FalkorDB, ognuno da valutare con la propria implementazione custom: `bfs`, `betweenness`, `labelPropagation`, `pageRank`, `SPpaths`, `SSpaths`, `WCC`, `harmonic`, `maxFlow`, `MSF`.



---

**Algorithm 2** Louvain

---

```
1: procedure LOUVAIN( $G = (V, E)$ )
2:   Inizializzare ogni nodo come una comunità separata
3:   repeat
4:     Per ogni nodo  $i$ , valutare il guadagno di modularità  $\Delta Q$ 
5:       ottenuto spostando  $i$  nella comunità del vicino  $j$ 
6:     Spostare  $i$  nella comunità che massimizza  $\Delta Q$  il vertice  $V_i$ 
7:   until nessun spostamento migliora la modularità
8:   Aggregare ogni comunità in un super-nodo
9:   Ripetere il processo sul grafo aggregato
10: end procedure
```

---

---

**Algorithm 3** Label Propagation

---

```
1: procedure LABELPROPAGATION( $G = (V, E)$ )
2:   Inizializzare ogni nodo con un'etichetta diversa  $L(v) \leftarrow v$ 
3:   repeat
4:     for ogni nodo  $v \in V$  nell'ordine random dei nodi do
5:       osservare le etichette dei suoi vicini  $N(v)$ 
6:       contare la frequenza di ciascuna etichetta tra i vicini
7:       assegnare a  $v$  l'etichetta più frequente tra quelle dei vicini, random se il massimo è condiviso
8:     end for
9:   until le etichette non cambiano più
10:  Raggruppare i nodi con la stessa etichetta
11:  Ogni gruppo ottenuto rappresenta una comunità
12:  return Partizioni  $\{C_1, \dots, C_k\}$  dove ogni  $C_i = \{v \in V \mid L(v) = i\}$ 
13: end procedure
```

---

## 5.6 Projections

In FalkorDB la nozione operativa di projection è legata alla chiave identificativa del grafo (key Redis) che ne determina il nome e conseguenti hash slot e shard di appartenenza. Il comando `GRAPH.QUERY` opera sempre su un singolo grafo. La selezione del sottografo è espressa direttamente nella query (pattern `MATCH/WHERE`) o nei parametri delle procedure algoritmo.

Questo approccio è coerente con l'architettura del modulo: il motore grafo è incorporato nel processo Redis, e le operazioni di storage e backend restano comandi Redis estesi (`GRAPH.QUERY`, `GRAPH.RO_QUERY`, `GRAPH.PROFILE`). Funzioni e algoritmi invece sono implementati nativamente in Rust.

## 5.7 Caricamento batch e parametri

Il caricamento usa batch in memoria: i dataframe vengono normalizzati, convertiti in record e processati con `UNWIND $rows`. Questa sintassi è scelta nel caso di inserimento di record, mantenendo la stessa logica di `MERGE` e aggiornamento proprietà. Il caricamento batch può usare `UNWIND $rows` e `MERGE` garantendo la possibilità di avere idempotenza su nodi e relazioni. Le query parametrizzate possono essere inviate con prefisso `CYPHER`.

Per dataset grandi la documentazione dei loader FalkorDB distingue infatti il batch loading dallo streaming: il loader Rust, per esempio, viene esplicitamente descritto come memory-efficient e capace di processare i dati senza caricare tutto in memoria.

Nel codice degli esperimenti in Python si utilizzano batch parametrizzati in memoria, costruiti a partire dai dataframe e inviati a FalkorDB tramite `UNWIND $rows`. Questa scelta è del tutto arbitraria e non requisito intrinseco del Database.

## 5.8 Effort e portabilità

La portabilità di una qualsiasi pipeline da Neo4j a FalkorDB non è banale e richiede un effort elevato. Pur essendo entrambi database a grafo, cambiano il modello operativo e il livello di esecuzione: Neo4j espone query e procedure del server grafo, mentre FalkorDB incapsula attraverso librerie GraphBLAS [3] [10] interrogando il backend Redis (`GRAPH.*`). Query e chiamate driver non sono direttamente sostituibili drop-in, quindi una parte del codice va riscritta e ri-validata. Non sempre poi esiste una corrispondenza diretta tra le stesse query. Scordiamoci quindi una "semplice sostituzione di server" in questo caso.

## 6 Scalare dati tabulari con Spark

Spark è un motore di esecuzione computazionale distribuito pensato per elaborare dati di dimensioni e tipo tali da non essere gestibili con metodi classici su un singolo nodo. Trova il suo spazio di utilizzo inizialmente come tool di trasformazione tabulare tramite la compatibilità SQL e dataframe. Spark legge, normalizza, aggrega e riscrive i dati. Il punto di forza non è soltanto la scalabilità, ma anche il modello dichiarativo delle trasformazioni, la valutazione lazy, la partizione del carico in micro batch e la possibilità di riutilizzare gli stessi passaggi in caso di fallimento o interruzione.

### 6.1 Feature principali

Spark viene utilizzato in genere con il linguaggio python tramite `pyspark`, che espone le API e la possibilità di interagire con il motore SQL locale o distribuito. Le operazioni più comuni sono `select`, `filter`, `withColumn`, `groupBy`, `agg`, `join` e `cache`. Il driver solitamente definisce il piano logico, mentre gli executor eseguono i task sulle partizioni; questo rende naturale sia il preprocessing batch sia l'analisi iterativa.

```
from pyspark.sql import functions as F
```

```
df.groupBy("Sesso").agg(F.avg("Eta").alias("eta_media")).show()
```

Questo tipo di aggregazione è tipico quando si vogliono produrre statistiche descrittive o altri output aggregati.

#### 6.1.1 Utilizzi comuni

La maggior parte delle funzionalità di base di una libreria per la data analysis è garantita e nativamente presente in Spark, come la conversione dei tipi, la rinomina delle colonne e l'utilizzo delle funzioni di slicing. Questi passaggi sono rilevanti quando i dati di partenza sono sporchi o eterogenei e devono essere resi coerenti prima dell'elaborazione finale. In particolare, Apache Spark consente di combinare trasformazioni locali e aggregazioni globali mantenendo il paradigma distribuito unendo i vari subtotali come risultati da presentare al nodo Driver.

### 6.2 Tipi di dato

La gestione a basso livello viene astratta usando il sistema dei tipi. Spark utilizza un backend da cui eredita la forte tipizzazione. I principali tipi di dato includono:

- tipi numerici, come `ByteType`, `ShortType`, `IntegerType`, `LongType`, `FloatType` e `DoubleType`;
- `DecimalType`, per valori decimali che richiedono una precisione controllata, ad esempio importi monetari;
- `StringType`, per dati testuali;
- `BooleanType`, per valori booleani;
- tipi temporali, come `DateType` e `TimestampType`;
- tipi complessi, come `ArrayType`, `MapType` e `StructType`;
- `BinaryType`, per dati binari.

#### 6.2.1 Rappresentazioni

A basso livello, ogni colonna di un `DataFrame` è associata a un tipo di dato definito ottenendo un campo omogeneo. Questa informazione è utilizzata da Spark durante la pianificazione e l'esecuzione delle operazioni.

La struttura logica dei dati è descritta attraverso uno *schema*, costituito dall'insieme dei nomi delle colonne e dei relativi tipi. Lo schema può essere dedotto automaticamente da Spark oppure definito esplicitamente. La definizione esplicita è particolarmente utile quando i dati provengono da sorgenti non uniformi, poiché evita che Spark interpreti erroneamente i valori, ad esempio identificando come stringa una colonna che dovrebbe contenere valori numerici.

La presenza di uno schema consente inoltre a Spark di conoscere in anticipo la struttura dei dati su cui dovranno essere eseguite le trasformazioni. Questo permette al motore di pianificazione di analizzare le operazioni richieste e di applicare ottimizzazioni prima dell'esecuzione effettiva.

I tipi complessi permettono di rappresentare strutture più articolate all'interno di una singola colonna. Un `ArrayType` rappresenta una sequenza di elementi dello stesso tipo, un `MapType` una collezione di coppie chiave-valore, mentre uno `StructType` permette di raggruppare più campi.

Durante l'esecuzione Spark può utilizzare rappresentazioni interne più compatte ed efficienti. Quando i dati devono essere trasferiti tra componenti del sistema, queste rappresentazioni vengono serializzate secondo un formato appropriato.

In particolare, Apache Parquet [6] è principalmente un formato di memorizzazione colonnare, utilizzato da Spark per leggere e scrivere dati in modo efficiente. Apache Arrow [5] svolge invece un ruolo importante nello scambio di dati in memoria.

Questa separazione tra livello logico e livello fisico è fondamentale per comprendere il funzionamento interno di Spark. Il sistema può infatti ragionare sui dati attraverso uno schema indipendente dalla loro rappresentazione a basso livello e, successivamente, scegliere modalità di memorizzazione, serializzazione e trasferimento più adatte alla specifica fase dell'elaborazione locale o distribuita.

### 6.2.2 Tidy data e formato wide/long

Un aspetto avanzato riguarda la gestione dei *tidy data*. In forma tidy ogni variabile ha una colonna dedicata, ogni osservazione occupa una riga e ogni valore è atomico. Sotto qualche esempio di pivoting e trasformazione tra formato wide e long.

```
df_tidy = df.selectExpr(
    "id",
    "stack(3, 'A', A, 'B', 'B', 'C', 'C')_as_(variabile, _valore)"
)

df_wide = (
    df_tidy
    .groupBy("id")
    .pivot("variabile")
    .agg(F.first("valore"))
)
```

## 6.3 Librerie avanzate

Apache spark può essere affiancato da librerie verticali per il calcolo come: - GraphX per il calcolo su grafi [7] - MLlib per il machine learning distribuito [8] - Spark Streaming per l'elaborazione di flussi in tempo reale [9]

Come vedremo successivamente queste librerie sono per ora una peculiarità di Spark.

```
spark
  .readStream
  .select($"value".cast("string").alias("jsonData"))
  .select(from_json($"jsonData", jsonSchema).alias("payload"))
  .writeStream
  .trigger("1_seconds")
  .start()
```

## 6.4 Effort di conversione da pandas a Spark DataFrame

L'utilizzo che è stato portato avanti in questo progetto riguarda un preprocessing tabulare banale, senza trasformazioni complesse. Ciò significa che il codice pandas esistente può essere convertito in Spark DataFrame, ma richiede un effort di riscrittura e validazione. Altri punti negativi dell'uso di spark sono l'eccessiva prolissità dei log che risultano difficili da interpretare, l'uso eccessivo di risorse di memoria (sia come dimensione che come bandwidth), CPU e ogni tipo di dispositivo che gli venga dato in pasto. Possiamo fare di meglio?

## 7 Usiamo Sail

Dove Spark con l'ecosistema Hadoop mostra segni di overread eccessivo, ad esempio con le classiche operazioni di computazione su dati tabulari, Sail si propone come alternativa più leggera e performante.

Per effettuare una connessione verso un cluster Spark si utilizza Spark Connect, che permette di indirizzare il client verso un server remoto.

```
spark = (SparkSession.builder
        .appName("FlightPreprocessSpark")
        .remote("sc://10.0.0.79:6066")
        .getOrCreate())
```

L'uso di `SparkSession` permette di mantenere una sintassi familiare per la creazione dei `DataFrame`, per le trasformazioni, il casting dei tipi, la gestione delle date, le operazioni *wide-to-long* tramite `stack`, la ricostruzione tramite `pivot` e l'espansione di array tramite `explode`. La compatibilità riguarda quindi soprattutto il livello `DataFrame` e consente di riutilizzare una parte del preprocessing tabulare senza riscrivere la logica delle trasformazioni.

## 7.1 Computing

La pipeline di esempio `05-GA-US-flight-2015_replace_neo4J_spark.ipynb` è trasformata utilizzando `Sail` prima del caricamento nel database a grafo. A questo punto `Sail` si comporta molto meglio rispetto a `Spark` intermini di tempo e utilizzo di risorse intese come tempo e memoria da assegnare alla computazione.

## 7.2 Benchmark della sessione Sail

Il benchmark nel file `06-diff-API.ipynb` solo una differente connessione mantenendo invariata la funzione di computazione. In questa computazione ed esperimento finale si vede espressamente il vantaggio di `Sail` rispetto a `Spark` in termini di efficienza e gestione delle risorse.

```
===== RISULTATO BENCHMARK =====
Spark : 25.537 s
Sail   : 0.586 s
Speedup Sail/Spark: 43.59x
```

## 7.3 Nota sull'integrazione nella pipeline

La parte `Spark/Sail` in questo progetto è usata per scalare il preprocessing tabulare mantenendo invariata l'interfaccia verso il caricamento grafo. `Sail` è adatto a questo replacement, mentre `Spark` rimane necessario per le librerie specializzate e le operazioni più complesse che richiedono l'intero ecosistema.

## 7.4 Limiti rispetto all'ecosistema Spark

`Sail` può offrire un'interfaccia compatibile per le operazioni `DataFrame`, ma non replica automaticamente l'intero ecosistema `Spark`. In particolare, le librerie specializzate utilizzate da `Spark` non sono disponibili in `Sail` con lo stesso livello di integrazione:

- **GraphX**: fornisce il modello a grafo e gli algoritmi distribuiti basati su `RDD`. `Sail` non offre l'implementazione `GraphX`; nel progetto le operazioni sul grafo restano quindi affidate al database grafo e alle sue API.
- **MLlib**: integra algoritmi di machine learning distribuito, come la regressione mostrata nel notebook `Spark`. La semplice compatibilità dei `DataFrame` non implica la disponibilità dei moduli `pyspark.ml` in `Sail`.
- **Spark Streaming**: permette di costruire query streaming, ad esempio leggendo socket e aggiornando conteggi con `readStream` e `writeStream`. `Sail` non fornisce la stessa infrastruttura di `Structured Streaming`; l'elaborazione dei flussi deve essere realizzata con componenti esterni o con un motore dedicato.

Di conseguenza `Sail` è una sostituzione mirata per il computing tabulare distribuito, non un rimpiazzo completo di `Spark`. Il vantaggio atteso è una maggiore leggerezza nelle operazioni `DataFrame`; il costo è rinunciare alle librerie verticali `GraphX`, `MLlib` e `Spark Streaming` e mantenere separati il calcolo tabulare, l'analisi del grafo e l'elaborazione degli eventi.

Potrebbe essere interessante integrare `spark` con `Sail` in modo da sfruttare i punti di forza di entrambi i sistemi: `Sail` per il computing tabulare distribuito leggero e `Spark` per le librerie specializzate e le operazioni più complesse. Questo setup però non è stato attualmente affrontato nel progetto.

## 8 Breve riepilogo

### 8.1 Backend Redis: CAP Theorem

Il file `stats.csv` riassume 36 esecuzioni del piano sperimentale, distribuite sui tre scenari `test_consistency`, `test_partitioning` e `test_availability`, con valori di `thread_count` pari a {1, 8, 32} e `local_test_count` pari a {1, 8, 32, 128}. In tutte le run il campo `exit_code` vale 0 e `result` vale ok, quindi non emergono fallimenti operativi né anomalie esplicite nei log.

Nel complesso, i risultati confermano che il protocollo sperimentale è ripetibile e che il cluster Redis ha mantenuto un comportamento stabile in tutte le configurazioni testate con un setup containerizzato: nel contesto osservato, i test sono stati eseguiti correttamente, il sistema di default garantisce consistenza e stabilità sufficienti per pensare di scalare senza pensare a problematiche di CAP perché già risolte.

In sintesi, il lavoro evidenzia la solidità del piano sperimentale e la sua ripetibilità, ma per trarre conclusioni più forti sarebbero necessari scenari di guasto più aggressivi, cluster più ampi e metriche aggiuntive sulla qualità delle risposte sotto partizione.

Sostanzialmente è un'ottima e affidabile base per poter creare pipeline veloci e scalabili.

### 8.2 Graph database Falkordb

FalkorDB ha mostrato di poter sostituire Neo4j nella pipeline sperimentale, mantenendo il modello a grafo e le principali operazioni di interrogazione attraverso i comandi implementati da FalkorDB. La sostituzione, tuttavia, non è drop-in: devono essere adattati il driver, la sintassi delle query, la gestione dei risultati e le procedure algoritmiche. Nel contesto Redis Cluster ogni grafo è associato a una singola chiave e quindi a un singolo shard, mentre le repliche mantengono una copia asincrona. FalkorDB risulta pertanto adatto a distribuire la computazione su repliche, ma richiede una fase di riscrittura e ri-validazione del codice prima di poter garantire la piena equivalenza con Neo4j. Neo4J da parte sua essendo semi-closed come richiede il pagamento di licenze e registrazioni centralizzate per l'uso del software.

### 8.3 Spark computing engine

Spark si è confermato un motore completo per Ogni computo relativo ai Big Data. Il modello DataFrame, lo schema tipizzato, la valutazione lazy e le operazioni di aggregazione e join permettono di costruire pipeline scalabili e riutilizzabili. Il principale vantaggio rispetto a soluzioni più semplici è l'ecosistema integrato che comprende GraphX, MLlib e Spark Structured Streaming. Questo risultato è accompagnato da un maggiore consumo di risorse e da una maggiore complessità operativa, particolarmente evidente per trasformazioni tabulari di piccole dimensioni.

### 8.4 Sail tabular computing

Sail è stato valutato come alternativa leggera per il calcolo tabulare distribuito, mantenendo una sintassi compatibile con Spark DataFrame e una connessione remota tramite Spark Connect. Nel benchmark considerato, la stessa computazione ha richiesto 25.537 secondi con Spark e 0.586 secondi con Sail, corrispondenti a uno speedup misurato di 43.59 volte. Il vantaggio riguarda quindi il preprocessing tabulare, mentre Sail (per ora) non sostituisce l'intero ecosistema Spark: non offre le integrazioni equivalenti a GraphX, MLlib e Spark Streaming. La scelta tra i due strumenti dipende pertanto dal carico: Sail per trasformazioni tabulari leggere e Spark quando sono necessarie le librerie specializzate.

## 9 Discussione e conclusioni

Questi 4 sotto-progetti sono stati pensati per affrontare diverse esigenze nel contesto dei Big Data, ciascuno con un focus specifico cercando per lo più di evitare vendor-lock-in. Quello che ha spinto alla comparazione tra questi strumenti è stato il desiderio di identificare le soluzioni più adatte a diversi scenari di calcolo distribuito, bilanciando prestazioni, complessità e correttezza dei risultati aggiungendo di volta in volta a piccoli step complessità. In sintesi non esiste una soluzione unica che domini su tutte, ma una combinazione di strumenti può essere la scelta più efficace a seconda del contesto e dei requisiti specifici.

## Riferimenti bibliografici

- [1] Brewer, E. A. (2000). Towards robust distributed systems (cap theorem). Keynote, PODC conference. Available at: <https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>.
- [2] FalkorDB (2026a). Falkordb. Accessed 2026-09-05.
- [3] FalkorDB (2026b). Falkordb. Accessed 2026-09-05.
- [4] FalkorDB (2026c). The theory and ideas behind falkordb. Accessed 2026-09-05.
- [5] Foundation, A. (2026a). Apache arrow — universal columnar format and multi-language toolbox for in-memory data. Accessed 2026-09-06.
- [6] Foundation, A. (2026b). Apache parquet — apache spark's columnar storage format. Accessed 2026-09-06.
- [7] Foundation, A. (2026c). Graphx — apache spark's graph processing library. Accessed 2026-09-06.
- [8] Foundation, A. (2026d). Mllib — apache spark's machine learning library. Accessed 2026-09-06.
- [9] Foundation, A. (2026e). Structured streaming — apache spark's stream processing library. Accessed 2026-09-06.
- [10] GraphBLAS (2026). Graphblas specification. Accessed 2026-06-19.
- [11] Jepsen (2026). Distributed systems testing. Accessed 2026-06-19.
- [12] Redis Labs (2024). Redis documentation. Accessed 2026-06-19.
- [13] Wikipedia (2026a). Label propagation algorithm — wikipedia, l'enciclopedia libera. Accessed 2026-09-06.
- [14] Wikipedia (2026b). Teorema cap — wikipedia, l'enciclopedia libera. [Online; in data 19-giugno-2026].